Machine Learning for Fitness Exercise Recognition

Classifying Barbell Exercises Using Wearable Sensor Data

Mohammed Eid, Ahmed Hossam, Mohammed Salah

April 17, 2026

Abstract

Wearable sensors have become an important tool for tracking physical activity and improving fitness performance. This project focuses on processing and analyzing accelerometer and gyroscope data collected during gym workouts. The goal is to build a machine learning system capable of classifying barbell exercises and counting repetitions. The workflow includes data preprocessing, visualization, outlier detection, feature engineering, machine learning model training, and repetition counting using signal processing techniques. Several models such as Naive Bayes, Support Vector Machines, Random Forest, and Neural Networks are evaluated to determine the most effective approach for exercise classification.

Keywords: Machine Learning, Wearable Sensors, Activity Recognition, Feature Engineering, Time Series Analysis

Contents

1 Introduction **4**
1.1 Background . . . 4
1.2 Problem Statement . . . 4
1.3 Project Objectives . . . 4
1.4 Dataset Overview . . . 5

2 Related Concepts **6**
2.1 Quantified Self . . . 6
2.2 Wearable Sensors . . . 7
2.2.1 Accelerometer . . . 7
2.2.2 Gyroscope . . . 8
2.3 Time Series Data . . . 9

3 Dataset Preparation **9**
3.1 Raw Dataset Description . . . 9
3.2 Reading Raw Data . . . 9
3.3 Data Cleaning . . . 10
3.4 Sensor Data Alignment . . . 10
3.5 Exporting the Processed Dataset . . . 11

4 Data Visualization **11**
4.1 Visualization Workflow . . . 11
4.2 Time Series Visualization . . . 11
4.3 Exercise Comparison . . . 12
4.4 Exporting Visualization Results . . . 13

5 Outlier Detection **13**
5.1 Outliers . . . 13
5.2 IQR Method . . . 13
5.3 Chauvenet's Criterion . . . 14
5.4 Local Outlier Factor . . . 15

6 Feature Engineering **15**
6.1 Overview . . . 15
6.2 Noise Filtering . . . 16
6.3 Principal Component Analysis . . . 16
6.4 Magnitude Features . . . 17
6.5 Temporal Features . . . 18
6.6 Frequency Features . . . 18
6.7 Clustering Features . . . 19

7 Machine Learning Models **19**
7.1 Training and Testing Split . . . 19
7.2 Feature Selection . . . 20
7.3 Models Used . . . 20
7.3.1 Naive Bayes . . . 21
7.3.2 Support Vector Machine . . . 21

7.3.3 Random Forest . . . 21
7.3.4 Neural Network . . . 21
7.4 Hyperparameter Tuning . . . 21
7.5 Model Evaluation . . . 21
7.5.1 Confusion Matrix . . . 22
7.5.2 Generalization Across Participants . . . 23

8 Repetition Counting **23**
8.1 Exercise Pattern Analysis . . . 23
8.2 Signal Filtering . . . 24
8.3 Repetition Counting Algorithm . . . 24
8.4 Evaluation . . . 25

9 Results and Discussion **26**
9.1 Best Model Performance . . . 26
9.2 Observations . . . 26
9.3 Limitations . . . 26

10 Conclusion **26**
10.1 Summary . . . 26
10.2 Future Work . . . 27

11 References **27**

1 Introduction

1.1 Background

The rapid growth of wearable technology has enabled individuals to continuously monitor various aspects of their daily lives. Devices equipped with motion sensors such as accelerometers and gyroscopes allow the collection of detailed movement data in real time. This has led to the emergence of the *Quantified Self* concept, where individuals track personal data related to physical activity, health, and behavior in order to gain insights and improve performance.

In the context of sports and fitness, wearable sensors provide an opportunity to automatically analyze workout activities. By capturing body movement during exercises, it becomes possible to apply machine learning techniques to recognize different physical activities and evaluate performance metrics. Such systems can assist athletes, coaches, and fitness enthusiasts in monitoring exercise quality, preventing injuries, and optimizing training routines.

Recent advances in machine learning and signal processing have made it feasible to extract meaningful information from raw sensor data. By analyzing patterns in accelerometer and gyroscope signals, machine learning models can learn to distinguish between different types of exercises and detect repetitions automatically.

1.2 Problem Statement

Tracking gym exercises manually can be time-consuming and error-prone. Many individuals rely on manual counting of repetitions and personal estimation of workout intensity. This approach often leads to inaccurate tracking and limited feedback regarding exercise performance.

The main challenge addressed in this project is the automatic recognition of barbell exercises using data collected from wearable motion sensors. Specifically, the project aims to develop a machine learning pipeline capable of analyzing accelerometer and gyroscope signals to classify different exercises performed during strength training sessions.

Additionally, the system should be able to estimate the number of repetitions performed during each exercise set. Achieving this goal requires several steps, including data preprocessing, visualization, outlier detection, feature engineering, and machine learning model training.

1.3 Project Objectives

The main objectives of this project are:

- Process raw accelerometer and gyroscope sensor data collected during gym workouts.
- Perform exploratory data analysis and visualization to better understand the characteristics of the motion signals.
- Detect and remove outliers in the dataset using statistical and machine learning methods.

- Engineer meaningful features from the raw sensor signals using temporal, frequency, and statistical techniques.
- Train and evaluate multiple machine learning models to classify different barbell exercises.
- Develop a custom algorithm capable of automatically counting exercise repetitions based on motion patterns.

1.4 Dataset Overview

The dataset used in this project was collected using the **MetaMotion** wearable motion sensor developed by MbientLab. The sensor is capable of measuring motion using an integrated accelerometer and gyroscope. These sensors record movement in three-dimensional space, making them suitable for analyzing physical activities such as weightlifting exercises.

The MetaMotion sensor is small, lightweight, and designed for wearable applications. It can be attached to the body or equipment to capture detailed motion data during workouts.

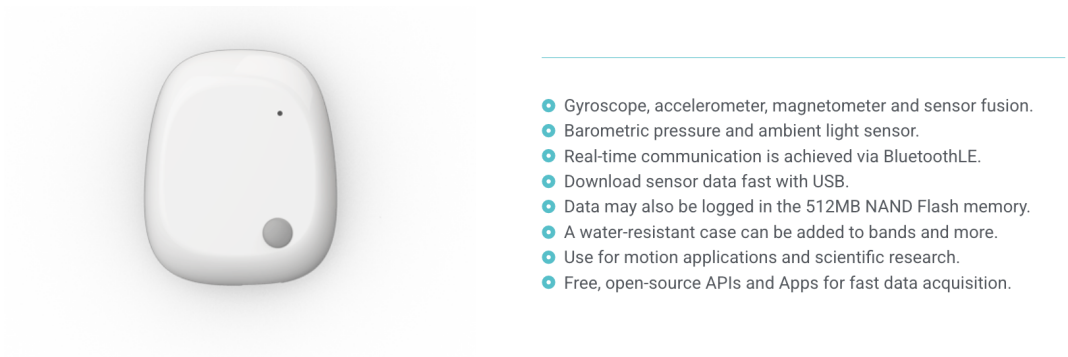

Figure 1: MetaMotion wearable motion sensor used for data collection.

The dataset contains recordings from multiple participants performing various barbell exercises at different intensity levels. During each workout session, the sensor recorded acceleration and angular velocity along three axes.

The dataset includes the following variables:

- **Accelerometer Data:** Acceleration measurements along the X, Y, and Z axes.
- **Gyroscope Data:** Angular velocity measurements along the X, Y, and Z axes.
- **Participant ID:** Identifier representing the participant performing the exercise.
- **Exercise Label:** The type of barbell exercise being performed.
- **Exercise Intensity:** The level of effort applied during the exercise (e.g., medium or heavy).
- **Set Number:** The specific set within the workout session.

An example representation of the recorded motion signals is shown in Figure 2.

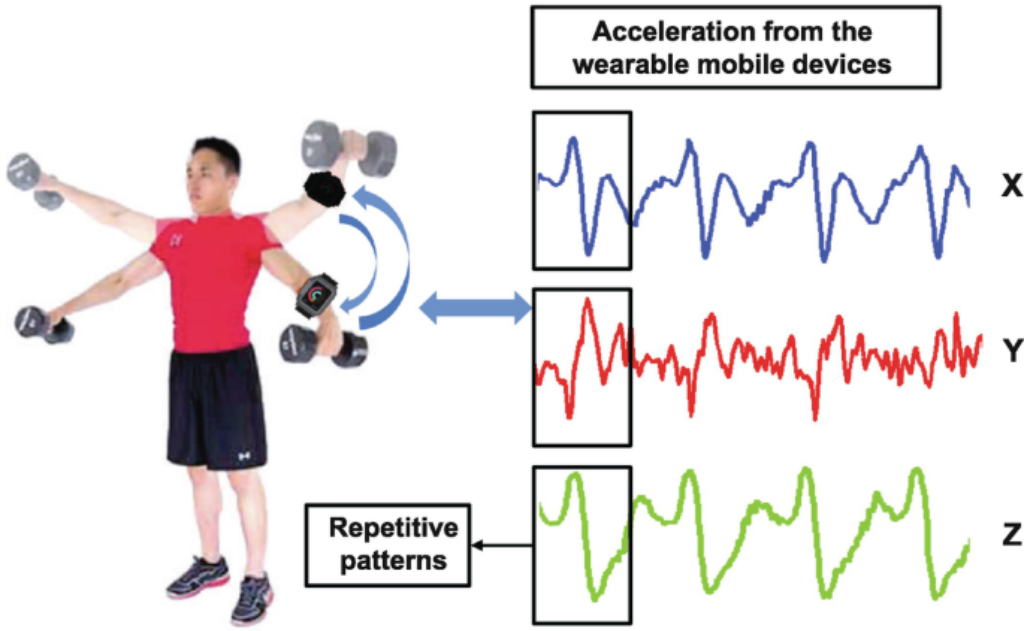

Figure 2: Example visualization of accelerometer time series signals recorded during an exercise.

These motion signals form the basis for the subsequent data analysis and machine learning pipeline developed in this project.

2 Related Concepts

2.1 Quantified Self

The *Quantified Self* (QS) movement refers to the practice of collecting and analyzing personal data in order to better understand human behavior, health, and performance. The concept emerged with the rise of wearable technology, mobile devices, and sensor-based systems that allow individuals to continuously monitor different aspects of their daily lives.

In the context of physical activity and fitness, the Quantified Self approach enables individuals to track metrics such as step count, heart rate, sleep quality, and exercise performance. These measurements can provide valuable insights into personal habits and help individuals optimize their lifestyle and training routines.

Machine learning plays an important role in the Quantified Self paradigm. By analyzing large amounts of sensor data collected over time, machine learning models can identify patterns in human movement and automatically recognize different activities. This capability has applications in health monitoring, sports analytics, rehabilitation, and personalized fitness systems.

The project presented in this report follows the Quantified Self approach by analyzing motion data collected from wearable sensors during gym workouts. The goal is to use this data to automatically classify barbell exercises and estimate the number of repetitions performed.

2.2 Wearable Sensors

Wearable sensors are small electronic devices designed to be worn on the body in order to collect physiological or motion-related data. These sensors typically contain multiple components such as accelerometers, gyroscopes, magnetometers, and wireless communication modules.

In fitness and sports applications, wearable sensors are used to track body movements and detect physical activities. The data generated by these sensors provides valuable information about motion patterns, which can be analyzed using signal processing and machine learning techniques.

In this project, motion data is collected using a MetaMotion wearable sensor that records both acceleration and rotational movement during exercise.

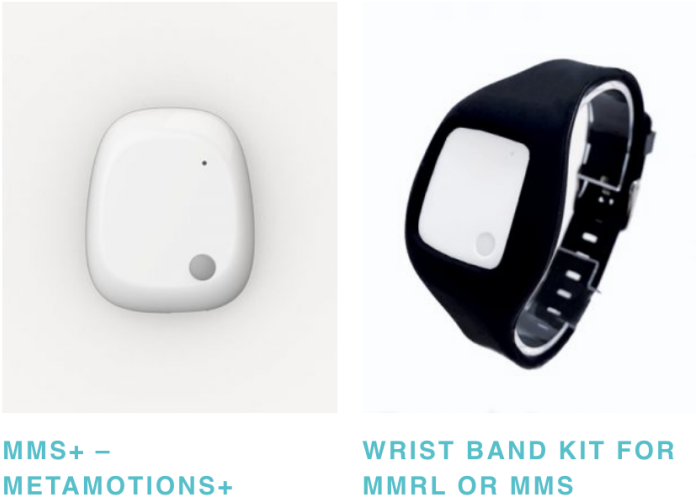

Figure 3: Example of a wearable motion sensor used for activity tracking.

Wearable motion sensors typically contain two main types of sensors that are particularly useful for analyzing physical activity: accelerometers and gyroscopes.

2.2.1 Accelerometer

An accelerometer is a sensor that measures acceleration forces acting on an object. These forces may be caused by gravity or by movement of the object itself. Accelerometers typically measure acceleration along three orthogonal axes (X, Y, and Z), allowing the detection of movement in three-dimensional space.

In the context of activity recognition, accelerometer data is used to capture patterns of motion such as lifting, walking, or running. For example, during a barbell exercise, the accelerometer can capture the vertical and horizontal movements of the barbell or the athlete's body. These signals can then be analyzed to identify specific exercises and detect repetition cycles.

Accelerometer measurements are usually expressed in units of gravitational acceleration (g), where one g is approximately equal to $9.8\,m/s^2$.

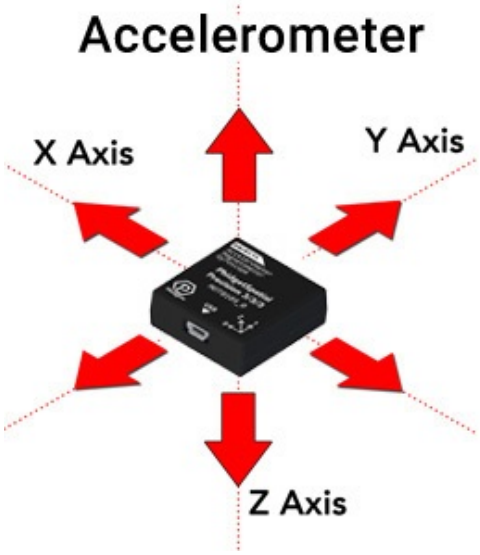

Figure 4: Accelerometer measurements along three spatial axes (X, Y, Z).

2.2.2 Gyroscope

A gyroscope is a sensor that measures angular velocity, which represents the rate of rotation around a particular axis. Similar to accelerometers, gyroscopes measure motion along three axes (X, Y, and Z). However, instead of measuring linear acceleration, they capture rotational movement.

Gyroscope data provides additional information about how an object rotates in space. This is particularly useful when analyzing complex movements such as weightlifting exercises, where both linear and rotational motions occur simultaneously.

For example, during certain barbell exercises, the body or the equipment may rotate slightly during each repetition. The gyroscope can capture these rotational patterns, which helps improve the accuracy of activity recognition models.

Angular velocity is typically measured in degrees per second ($^\circ/s$).

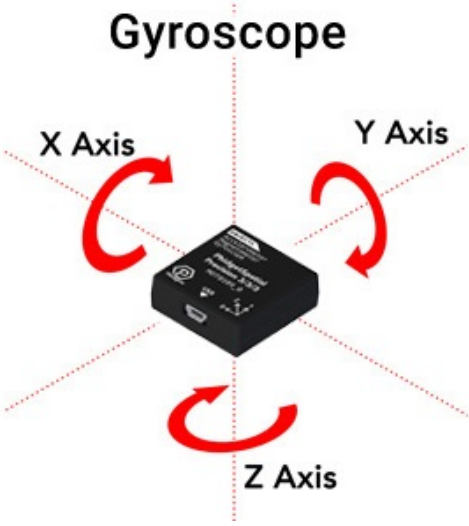

Figure 5: Gyroscope measurements capturing rotational movement around three axes.

2.3 Time Series Data

Sensor measurements collected from wearable devices are recorded sequentially over time. This type of data is known as *time series data*, where each observation is associated with a specific timestamp.

Time series data is commonly encountered in applications such as financial forecasting, weather monitoring, and physiological signal analysis. In the context of wearable sensors, time series data represents continuous recordings of motion signals produced during physical activities.

Analyzing time series data requires specialized techniques because the order and temporal relationships between observations are important. Methods such as filtering, feature extraction, and frequency analysis are often applied to extract meaningful information from these signals.

In this project, accelerometer and gyroscope measurements are recorded at a fixed sampling frequency during workout sessions. These signals form the basis for further data processing steps such as visualization, outlier detection, feature engineering, and machine learning model training.

3 Dataset Preparation

3.1 Raw Dataset Description

The raw dataset consists of multiple CSV files containing motion sensor measurements collected during different gym workout sessions. Each exercise session was recorded using a MetaMotion wearable sensor that captures both accelerometer and gyroscope data.

Each CSV file corresponds to a specific sensor type and exercise session. The filenames include useful information such as the participant identifier, the exercise type, and the exercise intensity level. For example, the filename structure encodes:

- Participant ID
- Exercise label (e.g., bench, squat, row)
- Exercise category (e.g., heavy or medium intensity)
- Sensor type (Accelerometer or Gyroscope)

The accelerometer data was recorded at a frequency of 12.5 Hz, while the gyroscope data was recorded at a frequency of 25 Hz. Because the sensors operate at different sampling frequencies, additional preprocessing was required to align the measurements.

3.2 Reading Raw Data

Python scripts were used to read all CSV files from the dataset directory and load them into pandas dataframes. The `glob` library was used to automatically detect all sensor files stored in the raw data folder.

Initially, individual files were loaded to inspect their structure and verify the available columns.

```python
import pandas as pd

data = pd.read_csv("sensor_data.csv")
```

After inspecting the file structure, a loop was implemented to iterate through all CSV files and load them into two separate dataframes: one for accelerometer measurements and another for gyroscope measurements.

During this step, additional metadata was extracted from the filename, including the participant ID, the exercise label, and the exercise category. These attributes were added as new columns to the dataframe in order to support later analysis and machine learning tasks.

3.3 Data Cleaning

Several preprocessing steps were applied to clean and organize the dataset.

First, unnecessary columns such as timestamps in milliseconds, elapsed time, and formatted time strings were removed because they were redundant after creating a proper datetime index.

Next, the epoch timestamp values were converted into Python datetime objects using the pandas `to_datetime()` function. These timestamps were then assigned as the dataframe index. Using a datetime index simplifies time-based analysis and enables easier alignment between different sensor measurements.

Finally, accelerometer and gyroscope datasets were combined into unified dataframes containing all recorded sessions.

3.4 Sensor Data Alignment

The accelerometer and gyroscope sensors record data at different sampling frequencies (12.5 Hz and 25 Hz respectively). Instead of performing traditional resampling, the datasets were aligned using a time-based merging approach.

The pandas function `merge_asof()` was used to match each accelerometer measurement with the closest gyroscope measurement based on their timestamps. This method preserves the temporal structure of the data while efficiently aligning measurements from both sensors.

```python
data_merged = pd.merge_asof(
    acc_df.iloc[:,:3].sort_index(),
    gyr_df.sort_index(),
    left_index=True,
    right_index=True
)
```

After merging the datasets, the columns were renamed to clearly represent the sensor signals:

- Accelerometer signals: `acc_x`, `acc_y`, `acc_z`
- Gyroscope signals: `gyr_x`, `gyr_y`, `gyr_z`

Additional metadata columns such as participant ID, exercise label, category, and set number were also retained in the final dataset.

3.5 Exporting the Processed Dataset

Once the accelerometer and gyroscope signals were successfully aligned, the processed dataset was saved as a serialized pandas object using the Pickle format. This format allows fast loading in later stages of the machine learning pipeline.

```
data_merged.to_pickle("../../data/interim/01_data_processed.pkl")
```

Saving the dataset in this format avoids the need to repeat the preprocessing steps when performing exploratory data analysis, feature engineering, or model training.

4 Data Visualization

Data visualization is an essential step in exploratory data analysis. In this project, visualization is used to understand motion patterns captured by the MetaMotion sensor during different barbell exercises. The accelerometer and gyroscope signals provide detailed information about the movement of the barbell over time. By visualizing these signals, it becomes possible to observe differences between exercises and identify patterns that can later be used for machine learning classification.

4.1 Visualization Workflow

The visualization process follows a structured workflow. First, the raw sensor data is loaded and processed to extract relevant metadata such as the participant identifier, exercise label, and training category. These attributes are extracted directly from the filenames of the sensor recordings.

Next, the accelerometer and gyroscope datasets are combined into a single dataset using time-based merging. This ensures that both sensor signals are synchronized according to their timestamps. After merging the datasets, the data is indexed using the timestamp column to allow time-series analysis.

Finally, visualizations are generated using the Matplotlib library. These figures allow us to inspect the sensor signals and explore relationships between exercises and motion patterns.

4.2 Time Series Visualization

Time-series visualization is used to analyze how sensor values change over time during exercise execution. Each exercise produces a sequence of accelerometer and gyroscope readings that correspond to the motion of the barbell.

The accelerometer measures linear acceleration in three axes (X, Y, and Z), while the gyroscope measures rotational movement around the same axes. By plotting these signals over time, it becomes possible to observe peaks, oscillations, and repeating patterns that correspond to repetitions of an exercise.

In this project, the accelerometer and gyroscope signals are plotted on separate axes within the same figure to allow clear comparison between linear and rotational motion.

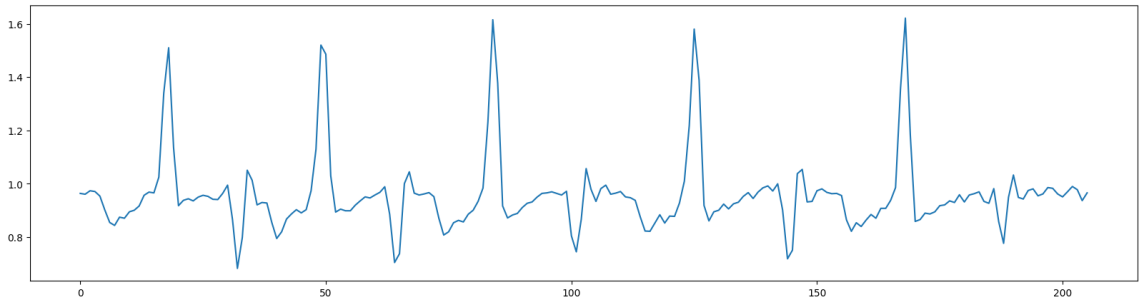

Figure 6: Example time-series visualization of accelerometer signals during a barbell exercise

These plots help reveal how motion intensity changes throughout the exercise and highlight repetitive patterns corresponding to individual repetitions.

4.3 Exercise Comparison

Different exercises produce distinct movement patterns in the sensor data. By comparing the time-series plots of different exercises, we can observe clear differences in signal behavior.

For example, exercises such as bench press and overhead press may produce sharp peaks in the acceleration signals due to explosive upward movement. In contrast, squats may show deeper and slower patterns corresponding to the lowering and lifting phases of the movement. These differences can be visually identified when plotting subsets of the sensor data.

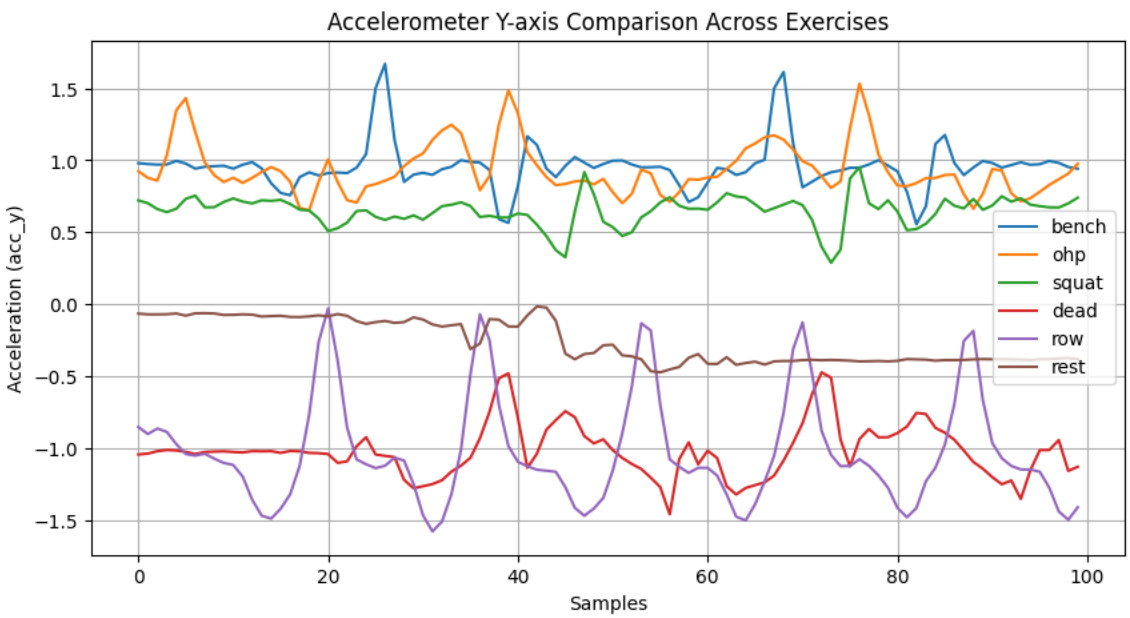

Figure 7: Comparison of accelerometer signals for different barbell exercises

These visual differences demonstrate that each exercise generates a unique motion signature in the sensor data. This observation is crucial for the next stages of the project, where machine learning algorithms will use these patterns to classify exercises automatically.

4.4 Exporting Visualization Results

After generating the figures, the plots are exported as high-resolution images. Exporting figures is important when preparing reports or sharing results with others. High-resolution images ensure that visual patterns remain clear and interpretable when included in documentation.

These exported visualizations serve as an important reference for understanding the dataset and validating the preprocessing steps before proceeding to outlier detection, feature engineering, and machine learning modeling.

5 Outlier Detection

In this stage of the project, we detect and remove extreme values from the sensor dataset. Outliers are observations that deviate significantly from the majority of the data and may introduce noise or negatively affect machine learning models. To identify such values, multiple statistical and machine learning techniques were applied to the accelerometer and gyroscope sensor data.

5.1 Outliers

Outliers are extreme values in a dataset that are much higher or lower than the majority of the observations. They can appear due to measurement errors, sensor noise, or unusual behavior in the recorded activity. In the context of this fitness tracking dataset, outliers may occur due to irregular movements or sensor inaccuracies. Identifying and handling these values is important because they can distort statistical measures and reduce model performance.

5.2 IQR Method

The Interquartile Range (IQR) method is a statistical technique used to detect outliers based on the distribution of the data. It relies on the first quartile ($Q1$) and the third quartile ($Q3$) of a dataset. The IQR is calculated as:

$$IQR = Q3 - Q1$$

Outliers are then defined as values outside the following bounds:

$$\text{Lower Bound} = Q1 - 1.5 \times IQR$$

$$\text{Upper Bound} = Q3 + 1.5 \times IQR$$

Any observation that falls outside these limits is considered an outlier. In this project, the IQR method was applied to each sensor column individually (e.g., accelerometer and gyroscope axes) to identify extreme values.

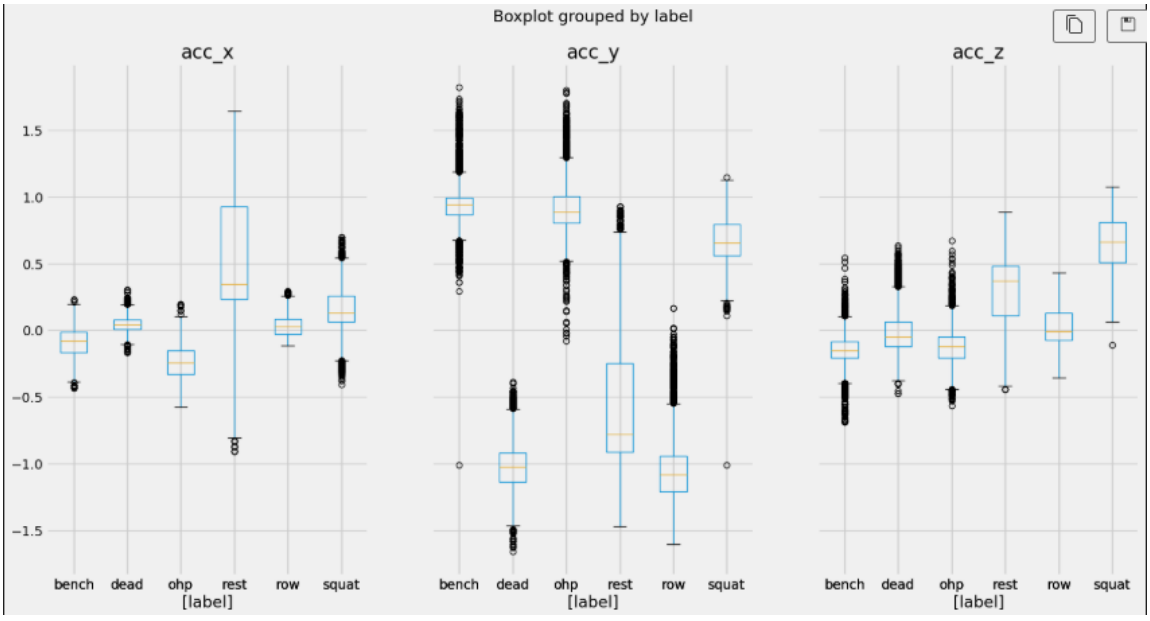

Figure 8: Outliers values in acc_{XYZ} directions using IQR Boxplot

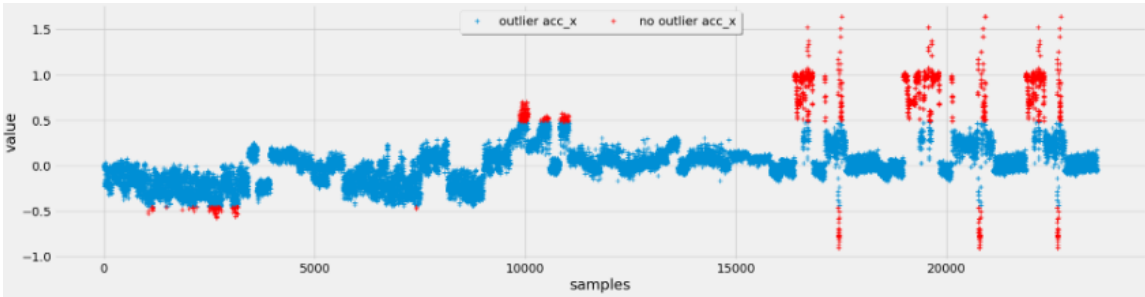

Figure 9: Outliers values in acc_X directions using IQR

5.3 Chauvenet's Criterion

Chauvenet's Criterion is another statistical approach used to detect outliers by considering the probability of observing a given data point in a normally distributed dataset. The method calculates the deviation of each observation from the mean in terms of standard deviations. If the probability of observing such a deviation is smaller than a predefined threshold, the value is considered an outlier.

The probability threshold is defined as:

$$P < \frac{1}{C \times N}$$

where:

- N is the number of observations in the dataset
- C is a constant (commonly set to 2)

This method assumes that the data follows a normal distribution. Therefore, histograms and box plots were used to visually check whether the sensor data approximately follows a bell-shaped distribution before applying this method.

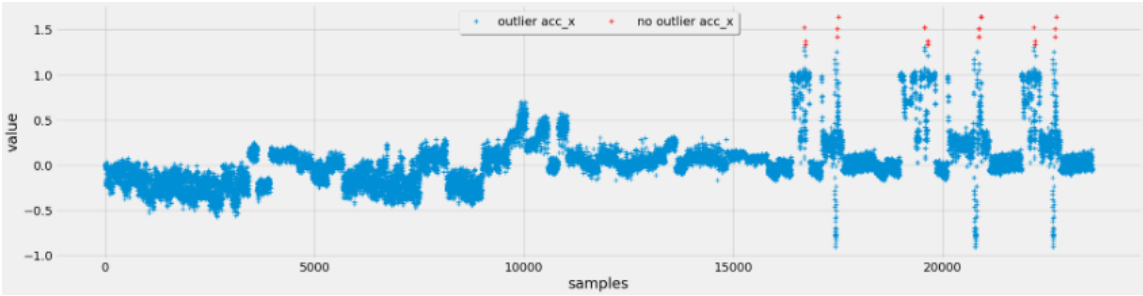

Figure 10: Outliers values in acc_x direction using Chauvene

5.4 Local Outlier Factor

The Local Outlier Factor (LOF) is a density-based anomaly detection algorithm. Unlike the previous methods, LOF is a machine learning approach that identifies outliers based on the local density of data points relative to their neighbors.

The algorithm compares the density of each observation to the densities of its nearest neighbors. If a point has a significantly lower density than its neighbors, it is considered an outlier.

In this project, the LOF algorithm from the `scikit-learn` library was used with 20 nearest neighbors. Instead of analyzing individual columns separately, LOF evaluates all selected sensor features simultaneously, allowing it to detect anomalies based on the combined behavior of multiple sensors.

This approach is particularly useful for multidimensional datasets where relationships between features may reveal abnormal observations.

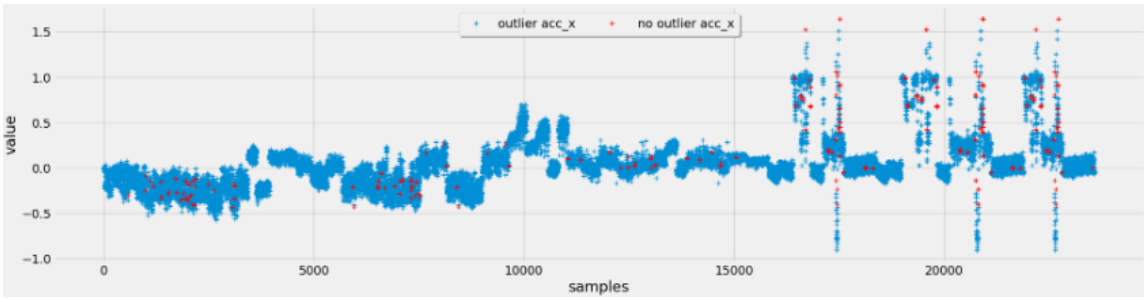

Figure 11: Outliers values in acc_x direction using LOF

6 Feature Engineering

6.1 Overview

Feature engineering is the process of transforming raw sensor data into informative variables that improve the performance of machine learning models. Since raw accelerometer and gyroscope signals are often noisy and complex, several transformation techniques were applied to extract meaningful patterns from the motion data.

The feature engineering pipeline consisted of the following steps:

- Noise filtering of raw signals
- Dimensionality reduction using Principal Component Analysis (PCA)

- Computation of signal magnitude features
- Extraction of temporal statistical features
- Extraction of frequency domain features using Fourier Transform
- Motion pattern grouping using clustering

These features help the machine learning model better distinguish between different types of barbell exercises.

6.2 Noise Filtering

Raw signals obtained from wearable sensors often contain high-frequency noise caused by sensor vibrations or environmental disturbances. To smooth the signals and retain the important motion patterns, a Butterworth low-pass filter was applied to the accelerometer and gyroscope data.

The Butterworth filter is commonly used in signal processing because it preserves the overall shape of the signal while removing rapid fluctuations.

Applying the low-pass filter results in smoother signals that better represent the underlying body movement during exercises.

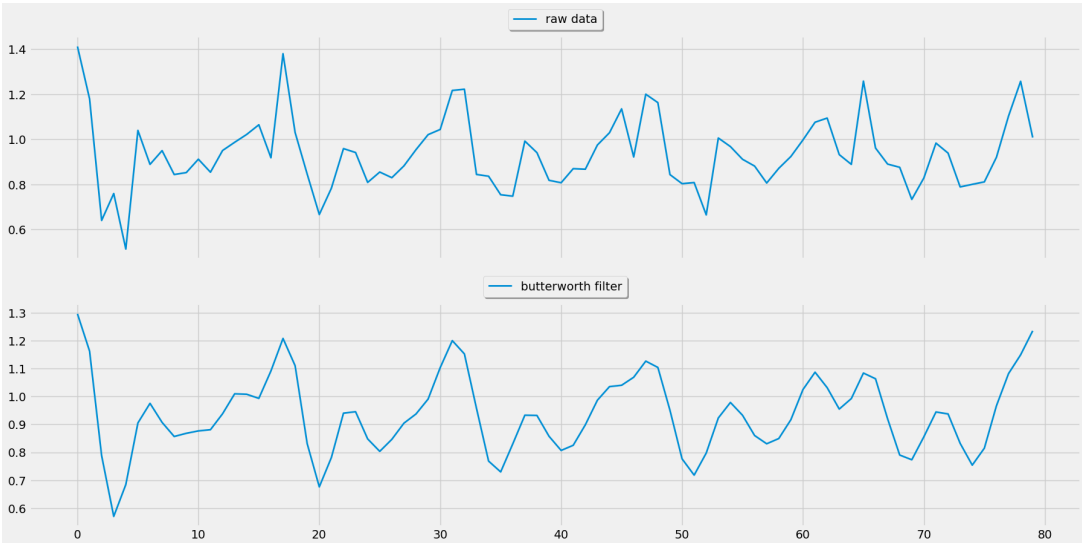

Figure 12: Example of accelerometer signal before and after applying a Butterworth low-pass filter.

6.3 Principal Component Analysis

Principal Component Analysis (PCA) was applied to reduce the dimensionality of the dataset while preserving most of the important information.

Since the accelerometer and gyroscope provide multiple correlated signals (X, Y, Z axes), PCA transforms these correlated variables into a smaller number of uncorrelated components called principal components.

These components capture the most significant variance in the data and help simplify the learning process for machine learning models.

Using PCA also helps reduce noise and redundancy in the dataset.

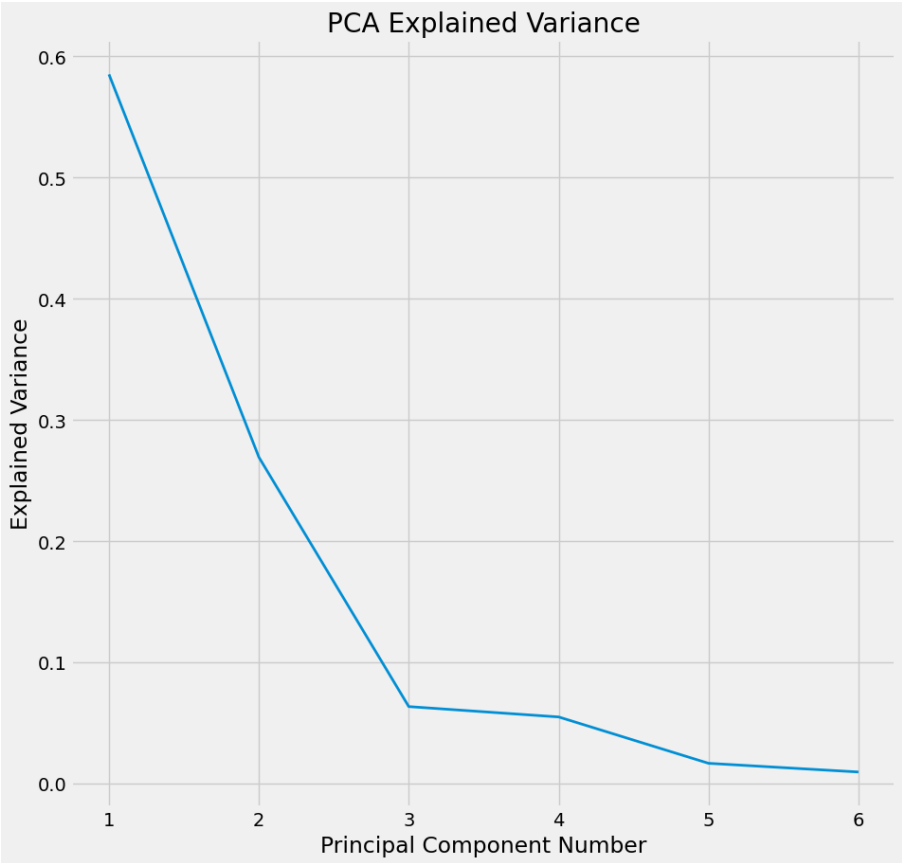

Figure 13: Explained variance ratio of principal components.

6.4 Magnitude Features

In addition to the individual axis measurements, the overall magnitude of motion was calculated for both the accelerometer and gyroscope signals.

The magnitude combines the three spatial axes into a single value representing the total movement intensity.

For the accelerometer signal, the magnitude is computed as:

$$Acc_{mag} = \sqrt{acc_x^2 + acc_y^2 + acc_z^2}$$

Similarly, the gyroscope magnitude is calculated as:

$$Gyr_{mag} = \sqrt{gyr_x^2 + gyr_y^2 + gyr_z^2}$$

These magnitude features help capture the total motion regardless of direction and are useful for distinguishing exercise types.

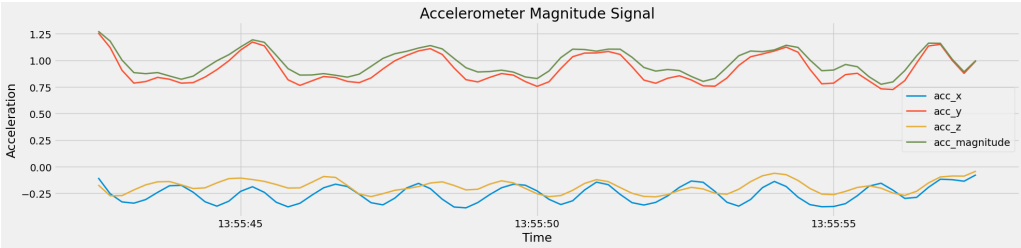

Figure 14: Example magnitude signal computed from accelerometer axes.

6.5 Temporal Features

Temporal features were extracted using rolling windows applied to the time series signals.

A rolling window calculates statistical measures over a fixed time interval, capturing how the signal evolves over time. Examples of temporal features include:

- Rolling mean
- Rolling standard deviation
- Rolling minimum
- Rolling maximum

These features capture patterns in the movement dynamics and help detect repetitive exercise motions.

6.6 Frequency Features

To analyze periodic patterns in exercise movements, the Discrete Fourier Transform (DFT) was applied to convert the signals from the time domain into the frequency domain.

Frequency domain features reveal dominant motion frequencies, which are particularly useful for identifying repetitive actions such as lifting repetitions.

Key frequency features include:

- Dominant frequency
- Frequency energy
- Spectral entropy

These features provide valuable information about the rhythm and periodicity of movements.

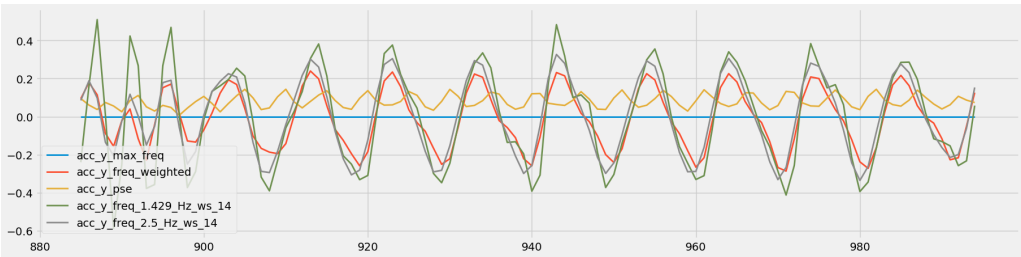

Figure 15: Frequency spectrum obtained using the Fourier Transform.

6.7 Clustering Features

Finally, clustering techniques were applied to identify groups of similar motion patterns within the dataset. K-means clustering was used to partition the sensor data into clusters representing different movement behaviors.

Clustering can reveal hidden structures in the motion data and help identify variations within exercises such as different lifting intensities.

The resulting cluster labels were added as additional features for the machine learning models.

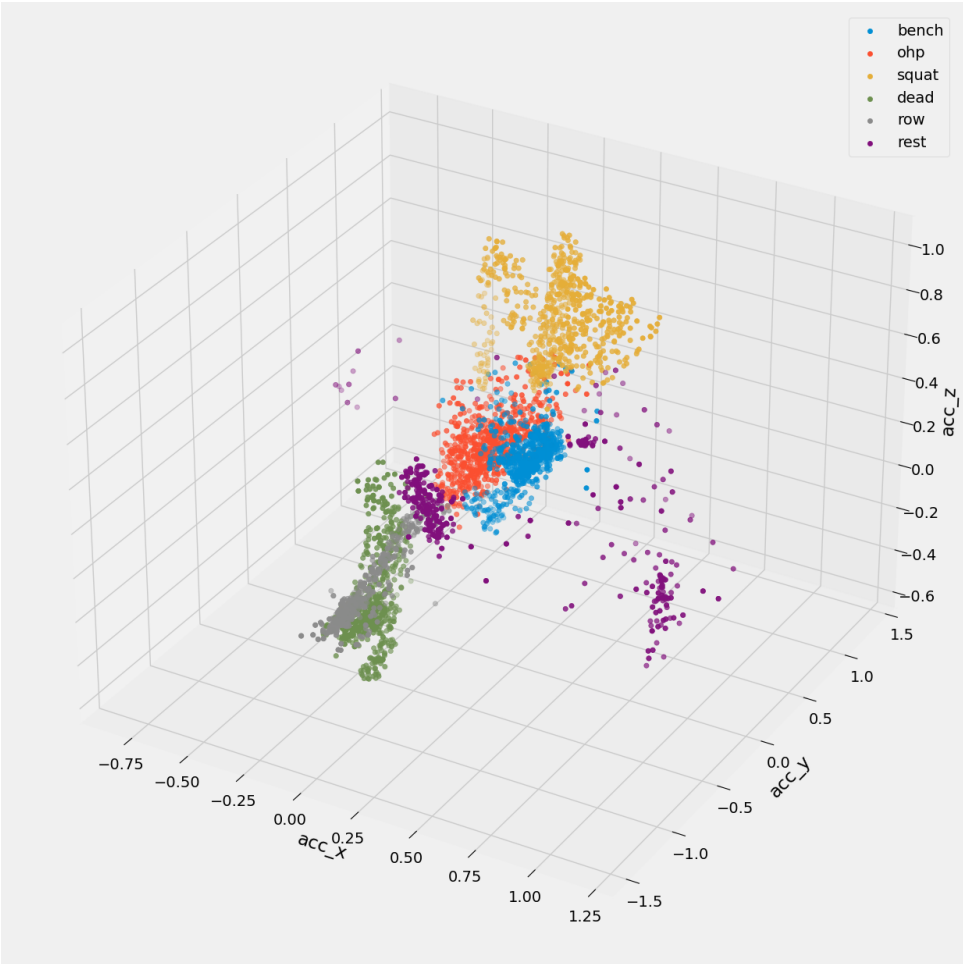

Figure 16: Example clustering of motion patterns using K-means.

7 Machine Learning Models

7.1 Training and Testing Split

The dataset was divided into training and testing subsets using a stratified sampling approach to preserve the class distribution of exercise labels.

First, non-relevant columns such as participant ID, category, and set number were removed. The remaining features were used as input variables, while the exercise label was used as the target variable.

The dataset was split using a 75% training and 25% testing ratio:

- Training set: used to train the models
- Test set: used to evaluate model performance

Stratification ensured that each exercise class was equally represented in both sets.

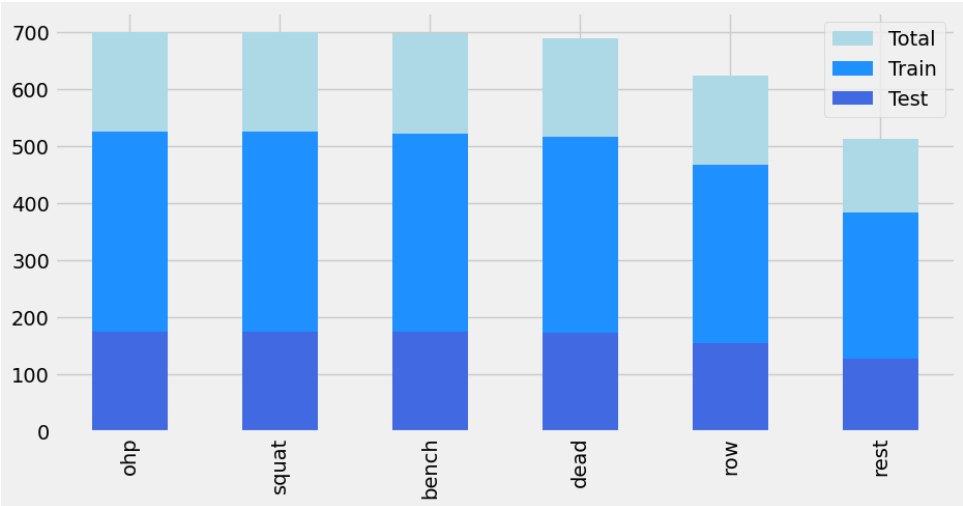

Figure 17: Distribution of labels in total, training, and testing datasets.

7.2 Feature Selection

Feature selection was performed using forward feature selection with a simple decision tree model. This method incrementally selects features that improve classification accuracy.

The process evaluated up to 10 features and ranked them based on their contribution to model performance.

The final selected features were mainly derived from frequency domain features, indicating that frequency characteristics are highly informative for distinguishing exercise types.

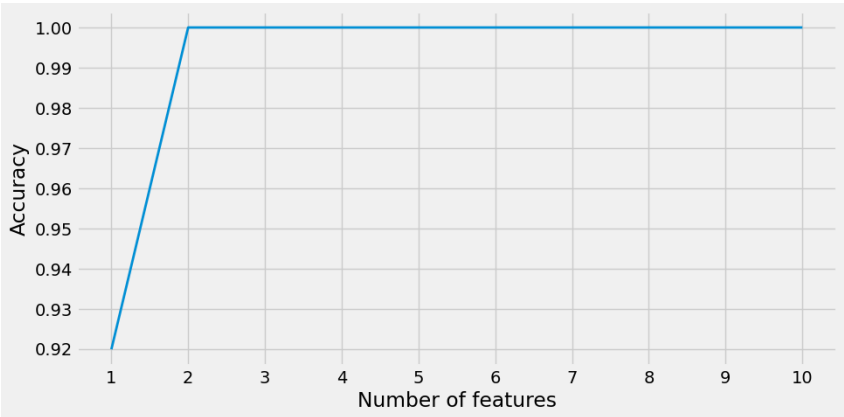

Figure 18: Forward feature selection showing accuracy improvement as features are added.

7.3 Models Used

Several machine learning models were trained and evaluated on different feature sets.

7.3.1 Naive Bayes

Naive Bayes is a probabilistic classifier based on Bayes' theorem. It assumes independence between features and provides a simple yet effective baseline model.

7.3.2 Support Vector Machine

Support Vector Machines (SVM) are powerful classifiers that aim to find the optimal hyperplane that separates classes in high-dimensional space. Although not explicitly implemented in the final pipeline, similar deterministic models such as K-Nearest Neighbors and Decision Trees were evaluated.

7.3.3 Random Forest

Random Forest is an ensemble learning method that constructs multiple decision trees and combines their predictions. It is robust to noise and performs well on complex datasets.

7.3.4 Neural Network

A feedforward neural network was used as a non-linear model capable of capturing complex relationships in the data. Due to its stochastic nature, multiple runs were averaged to obtain stable performance estimates.

7.4 Hyperparameter Tuning

Hyperparameter tuning was performed using grid search for several models, including Random Forest, K-Nearest Neighbors, and Decision Trees.

Grid search evaluates different parameter combinations and selects the configuration that maximizes model performance on the validation set.

For non-deterministic models such as neural networks and random forests, multiple iterations were performed and averaged to ensure reliable results.

7.5 Model Evaluation

Models were evaluated using classification accuracy on the test dataset. Additionally, confusion matrices were used to analyze the classification performance for each exercise class.

A grouped bar plot was created to compare the performance of different models across multiple feature sets.

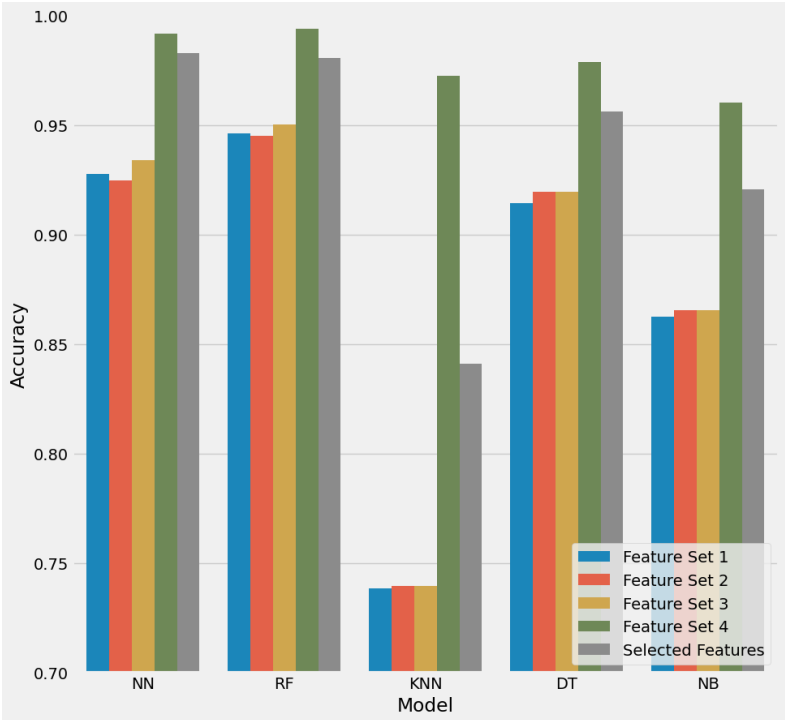

Figure 19: Comparison of model accuracy across different feature sets.

The Random Forest model achieved the best overall performance and was selected as the final model.

7.5.1 Confusion Matrix

To further evaluate the model, a confusion matrix was generated to visualize the classification results.

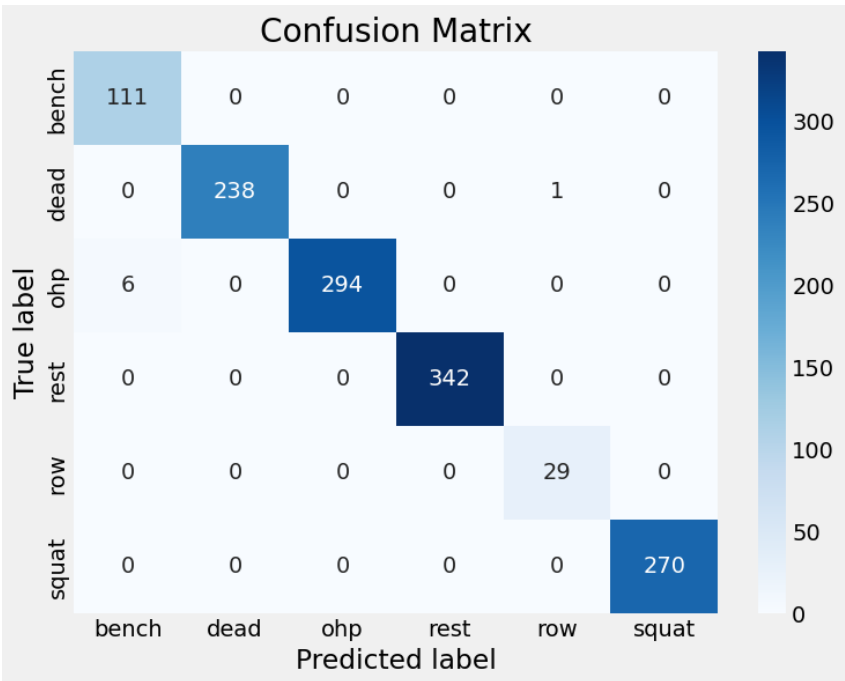

Figure 20: Confusion matrix of the best performing model.

7.5.2 Generalization Across Participants

To evaluate model robustness, an additional experiment was conducted where the model was trained on data from all participants except one and tested on the unseen participant.

This approach simulates real-world conditions where the model must generalize to new users.

The Random Forest model maintained strong performance, demonstrating its ability to generalize across different individuals performing the same exercises.

Model	FS1	FS2	FS3	FS4	Selected
Neural Network	0.931	0.928	0.924	**0.987**	0.977
Random Forest	0.951	0.950	0.952	**0.995**	0.980
KNN	0.738	0.739	0.739	**0.973**	0.841
Decision Tree	0.909	0.910	0.919	**0.985**	0.961
Naive Bayes	0.863	0.866	0.866	**0.958**	0.921

Table 1: Model accuracy comparison across different feature sets

The results clearly show that Feature Set 4, which includes all engineered features (temporal, frequency, and clustering), provides the highest accuracy across all models.

Random Forest and Neural Network achieved the best performance with an accuracy close to 99%, demonstrating their ability to capture complex patterns in the sensor data.

KNN performed poorly on basic feature sets but significantly improved when advanced features were included, highlighting the importance of feature engineering.

Overall, feature engineering had a greater impact on performance than the choice of model itself.

8 Repetition Counting

8.1 Exercise Pattern Analysis

The motion sensor signals exhibit periodic patterns that correspond to individual exercise repetitions. By visualizing accelerometer and gyroscope signals for a single set, it is possible to observe clear repeating peaks and valleys that represent the movement cycles of each repetition.

To better understand these patterns, multiple signals such as `acc_x`, `acc_y`, `acc_z`, and their magnitude `acc_r` were plotted. Similarly, gyroscope signals were analyzed.

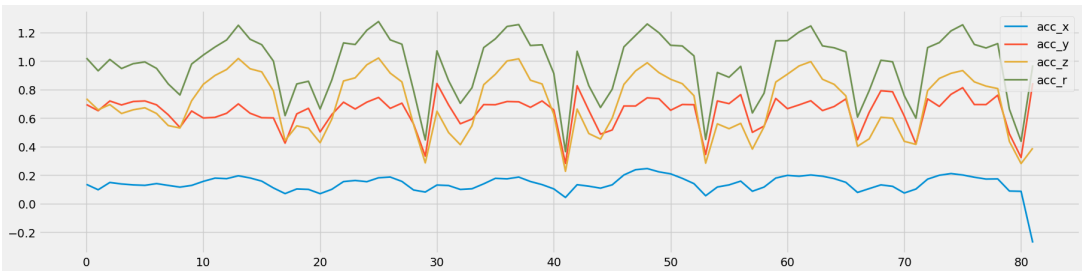

Figure 21: Example of raw sensor signals showing repetitive patterns during an exercise set

8.2 Signal Filtering

Raw sensor data contains noise that makes it difficult to clearly identify repetitions. To address this, a Butterworth low-pass filter was applied to smooth the signals and highlight the underlying motion pattern.

The filter was applied using a sampling frequency of 5 Hz and different cutoff frequencies depending on the exercise type. For example:

- Bench press: cutoff = 0.4 Hz
- Squat: cutoff = 0.35 Hz
- Row: cutoff = 0.65 Hz

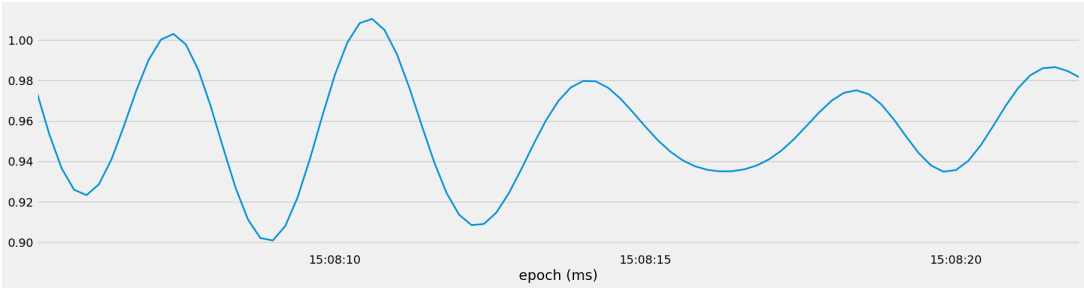

Figure 22: Comparison between raw signal and filtered signal showing clearer repetition patterns

8.3 Repetition Counting Algorithm

A custom algorithm was developed to automatically count repetitions based on the filtered signal.

The main steps of the algorithm are:

1. Apply a low-pass filter to the selected signal (e.g., `acc_r` or `gyr_x`)
2. Detect local maxima (peaks) using `argrelextrema`
3. Count the number of peaks, where each peak represents one repetition

The implementation is shown below:

```python
def count_reps(dataset, cutoff=0.4, order=10, column="acc_r"):
    data = LowPass.low_pass_filter(
        dataset, col=column, sampling_frequency=fs,
           cutoff_frequency=cutoff, order=order
    )

    indexes = argrelextrema(data[column + "_lowpass"].values, np.
       greater)
    peaks = data.iloc[indexes]

    return len(peaks)
```

Different exercises required different configurations:

- Squat: lower cutoff frequency (0.35 Hz)
- Row: used gyroscope signal (`gyr_x`) instead of accelerometer
- Other exercises: used `acc_r` signal

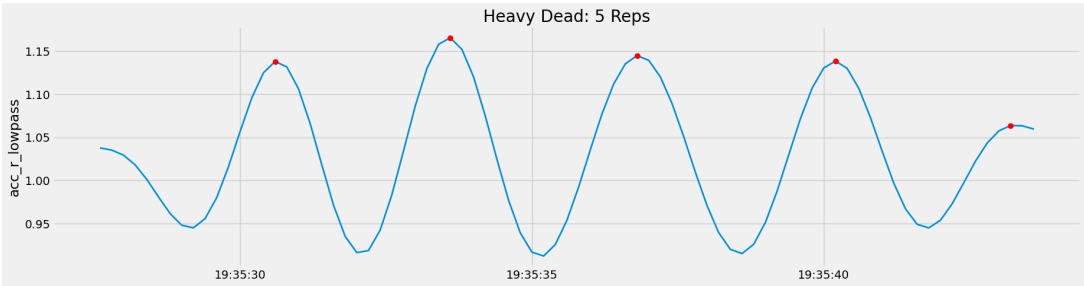

Figure 23: Detected peaks corresponding to repetitions in the filtered signal

8.4 Evaluation

To evaluate the performance of the repetition counting algorithm, a benchmark dataset was created. The ground truth number of repetitions was defined based on the exercise intensity:

- Heavy sets: 5 repetitions
- Medium/light sets: 10 repetitions

The predicted repetitions were compared against the ground truth using Mean Absolute Error (MAE):

```
error = mean_absolute_error(rep_df["reps"], rep_df["reps_pred"]).
    round(2)
```

Additionally, the average predicted and actual repetitions per exercise were visualized:

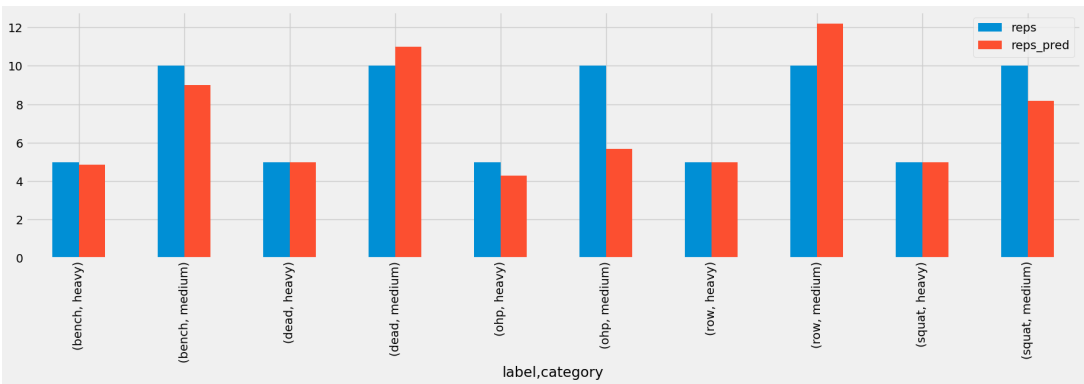

Figure 24: Comparison between actual and predicted repetitions across exercises

Bar chart from:

```
rep_df.groupby(["label", "category"])["reps", "reps_pred"].mean()
    .plot.bar()
```

The results showed that the algorithm is capable of accurately estimating repetitions for most exercises, although small deviations may occur due to noise, variation in movement speed, and differences between participants.

9 Results and Discussion

9.1 Best Model Performance

The Random Forest model achieved the highest performance among all tested models, reaching an accuracy close to 99% when using the full feature set (Feature Set 4). Neural Networks also demonstrated competitive performance, indicating that complex models benefit significantly from the engineered features.

9.2 Observations

The results highlight the importance of feature engineering in this project. Features extracted from temporal, frequency, and clustering techniques significantly improved classification performance compared to using raw sensor data alone.

Additionally, different exercises exhibited distinct motion patterns in both accelerometer and gyroscope signals, which made them separable using machine learning models.

9.3 Limitations

- Limited number of participants (only a small group was included)
- Dataset size is relatively small for robust generalization
- Variability in sensor placement may affect signal consistency

10 Conclusion

10.1 Summary

This project demonstrated how machine learning techniques can be applied to classify barbell exercises using wearable sensor data. Starting from raw accelerometer and gyroscope signals, the pipeline included data preprocessing, feature engineering, model training, and evaluation.

The results showed that combining advanced feature engineering with powerful models such as Random Forest leads to highly accurate classification performance.

The full implementation of the project is available at: `https://github.com/M0hamed-Eid/FitnessTracker.git`

10.2 Future Work

Future improvements for this project include:

- Collecting more data from a larger and more diverse group of participants
- Deploying the model as a real-time application (e.g., web or mobile app)
- Improving robustness for different sensor placements and environments

11 References

- Hoogendoorn, M., & Funk, B. (2018). *Machine Learning for the Quantified Self.*
- ML4QS Project Website: `https://ml4qs.org`
- MetaMotion Sensor: `https://mbientlab.com/metamotions/`